IMPLEMENTASI PENGUJIAN PERANGKAT LUNAK

Tugas Materi 4

Implementasi Unit Testing dengan GitHub Actions

Disusun oleh :

Cindy Yulianti NIM : 1224606

STIMK BANDUNG

Jl. Cikutra No. 133 Cikutra, Kec. Cibeunying Kidul,

Kota Bandung

2026

Bagian A : Git & Github Dasar

Soal 1

1. Apa perbedaan antara Centralized VCS dan Distributed VCS? Berikan masing- masing 2 contoh!

   Perbedaan utama antara Centralized VSC dan Distributed VCS terletak pada penyimpanan repositori. Centralized VCS adalam sistem kontrol versi yang menyimpan seluruh riwayat perubahan pada satu sever pusat.

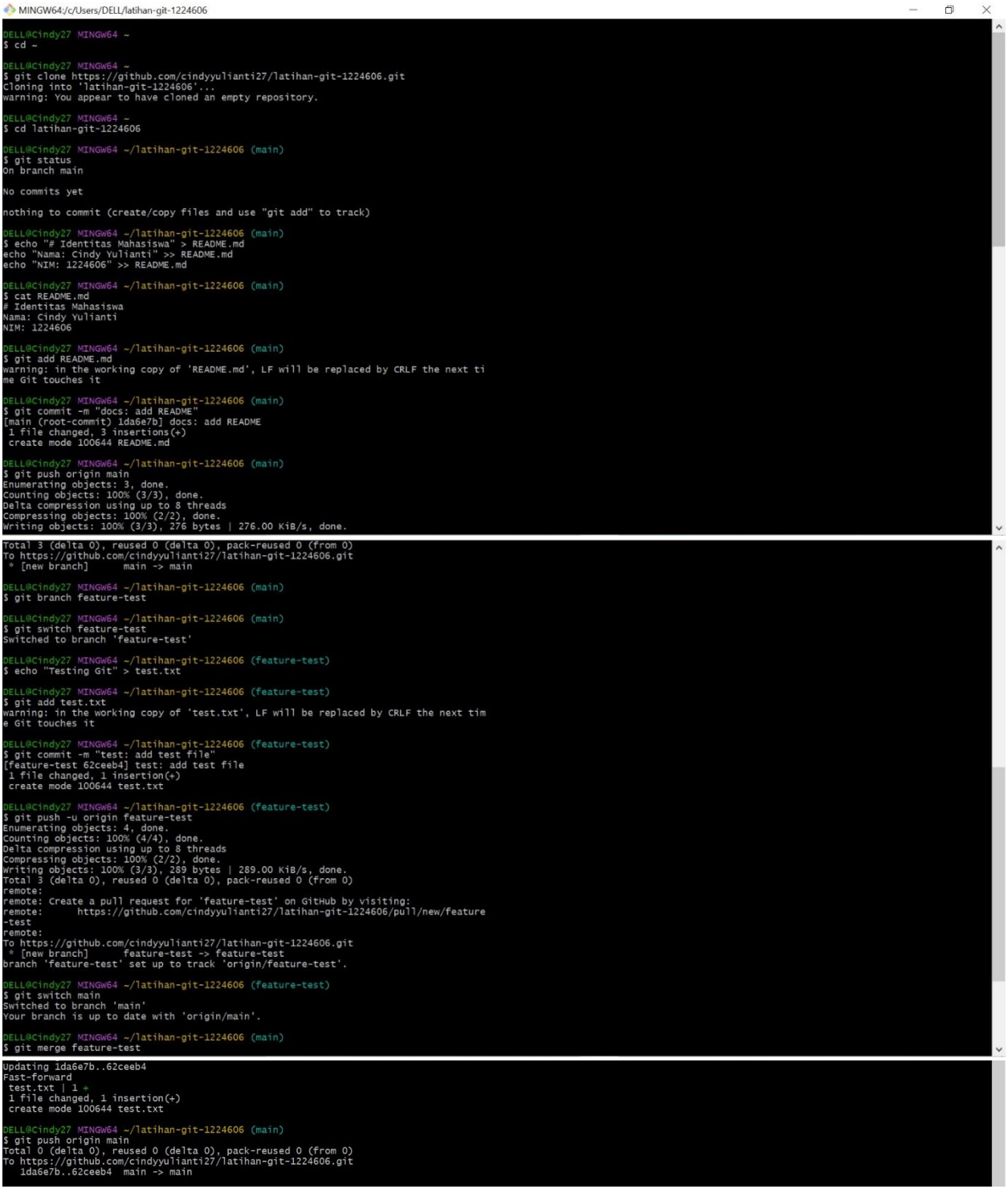
   Developer perlu terhubung ke server untuk melakukan perubahan, dan sistem ini bergantung pada server pusat maka jika server gangguan, pekerjaan tertentu bisa terhambat. Contoh dari Centralized VCS adalah SVN dan CVS.

   Sedangkan, Distributed VCS memungkinkan setiap developer memiliki salinan repositori dan riwayat lengkap proyek di hard drive sendiri. Sistem ini tidak selalu bergantung pada server pusat, developer dapat bekerja secara offline dan menyinkronkan perubahan agar pekerjaan dapat terhubung. Contoh dari Distributed VCS adalah Git dan Mercurial.
2. Sebutkan 5 Git commands yang wajib dikuasai beserta fungsinya!
   1) Git inti berfungsi untuk inisialisasi repository Git baru di folder lokal
   2) Git add berfungsi untuk menambahkan file ke staging area
   3) Git commit berfungsi untuk menyimpan perubahan dengan pesan commit
   4) Git push berfungsi mengirim commit lokal ke remote repository
   5) Gitt pull berfungsi mengambil perubahan dari remote ke lokal
3. Apa fungsi .gitignore dalam repository Git?

   File .gitignore adalah file yang berfungsi untuk menjaga repository tetap rapi dengan memberi tahu Git agar mengabaikan file hasil build, file konfigurasi IDE, dan file lain yang tidak diperlukan agar tidak ikut di-commit ke repository Git.

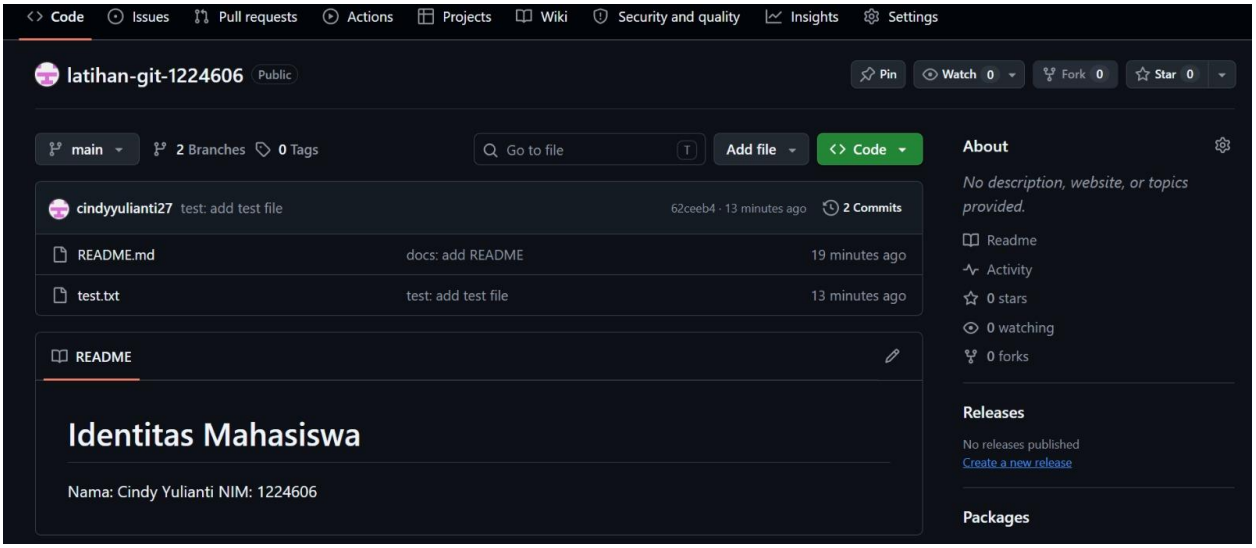
Soal 2

Praktik Git Commands:



Hasil latihan Git pada Github

Link repository : https://github.com/cindyyulianti27/latihan-git-1224606.git



Command yang digunakan :

1. Clone Repository

```
DELL@Cindy27 MINGW64 ~
$ git clone https://github.com/cindyyulianti27/latihan-git-1224606.git
Cloning into 'latihan-git-1224606'...
warning: You appear to have cloned an empty repository.

DELL@Cindy27 MINGW64 ~
$ cd latihan-git-1224606

DELL@Cindy27 MINGW64 ~/latihan-git-1224606 (main)
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

2. Membuat README.md

```
DELL@Cindy27 MINGW64 ~/latihan-git-1224606 (main)
$ echo "# Identitas Mahasiswa" > README.md
echo "Nama: Cindy Yulianti" >> README.md
```

```
echo "NIM: 1224606" >> README.md

DELL@Cindy27 MINGW64 ~/latihan-git-1224606 (main)
$ cat README.md
# Identitas Mahasiswa
Nama: Cindy Yulianti
NIM: 1224606
```

3. Commit README.md

```
DELL@Cindy27 MINGW64 ~/latihan-git-1224606 (main)
$ git add README.md
warning: in the working copy of 'README.md', LF will be
replaced by CRLF the next time Git touches it

DELL@Cindy27 MINGW64 ~/latihan-git-1224606 (main)
$ git commit -m "docs: add README"
[main (root-commit) 1da6e7b] docs: add README
 1 file changed, 3 insertions(+)
 create mode 100644 README.md
```

4. Push README.md ke GitHub

```
DELL@Cindy27 MINGW64 ~/latihan-git-1224606 (main)
$ git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 276 bytes | 276.00 KiB/s,
done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from
0)
To https://github.com/cindyyulianti27/latihan-git-
1224606.git
 * [new branch]      main -> main
```

5. Membuat Branch Baru dengan Nama feature-test

```
DELL@Cindy27 MINGW64 ~/latihan-git-1224606 (main)
$ git branch feature-test
```

6. Berpindah ke Branch feature-test

```
DELL@Cindy27 MINGW64 ~/latihan-git-1224606 (main)
$ git switch feature-test
Switched to branch 'feature-test'
```

7. Membuat File test.txt dan Melakukan Commit

```
DELL@Cindy27 MINGW64 ~/latihan-git-1224606 (feature-test)
$ echo "Testing Git" > test.txt
```

```
DELL@Cindy27 MINGW64 ~/latihan-git-1224606 (feature-test)
$ git add test.txt
warning: in the working copy of 'test.txt', LF will be
replaced by CRLF the next time Git touches it

DELL@Cindy27 MINGW64 ~/latihan-git-1224606 (feature-test)
$ git commit -m "test: add test file"
[feature-test 62ceeb4] test: add test file
 1 file changed, 1 insertion(+)
 create mode 100644 test.txt
```

8. Push Branch feature-test ke GitHub

```
DELL@Cindy27 MINGW64 ~/latihan-git-1224606 (feature-test)
$ git push -u origin feature-test
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 289 bytes | 289.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature-test' on GitHub by
visiting:
remote:      https://github.com/cindyyulianti27/latihan-git-
1224606/pull/new/feature-test
remote:
To           https://github.com/cindyyulianti27/latihan-git-
1224606.git
 * [new branch]      feature-test -> feature-test
branch 'feature-test' set up to track 'origin/feature-test'.
```

9. Berpindah ke Branch main

```
DELL@Cindy27 MINGW64 ~/latihan-git-1224606 (feature-test)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

10. Merge Branch feature-test ke main

```
DELL@Cindy27 MINGW64 ~/latihan-git-1224606 (main)
$ git merge feature-test
Updating 1da6e7b..62ceeb4
Fast-forward
 test.txt | 1 +
```

```
 1 file changed, 1 insertion(+)
 create mode 100644 test.txt
```

11. Push Hasil Merge ke GitHub

```
DELL@Cindy27 MINGW64 ~/latihan-git-1224606 (main)
$ git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To           https://github.com/cindyyulianti27/latihan-git-
1224606.git
   1da6e7b..62ceeb4  main -> main
```

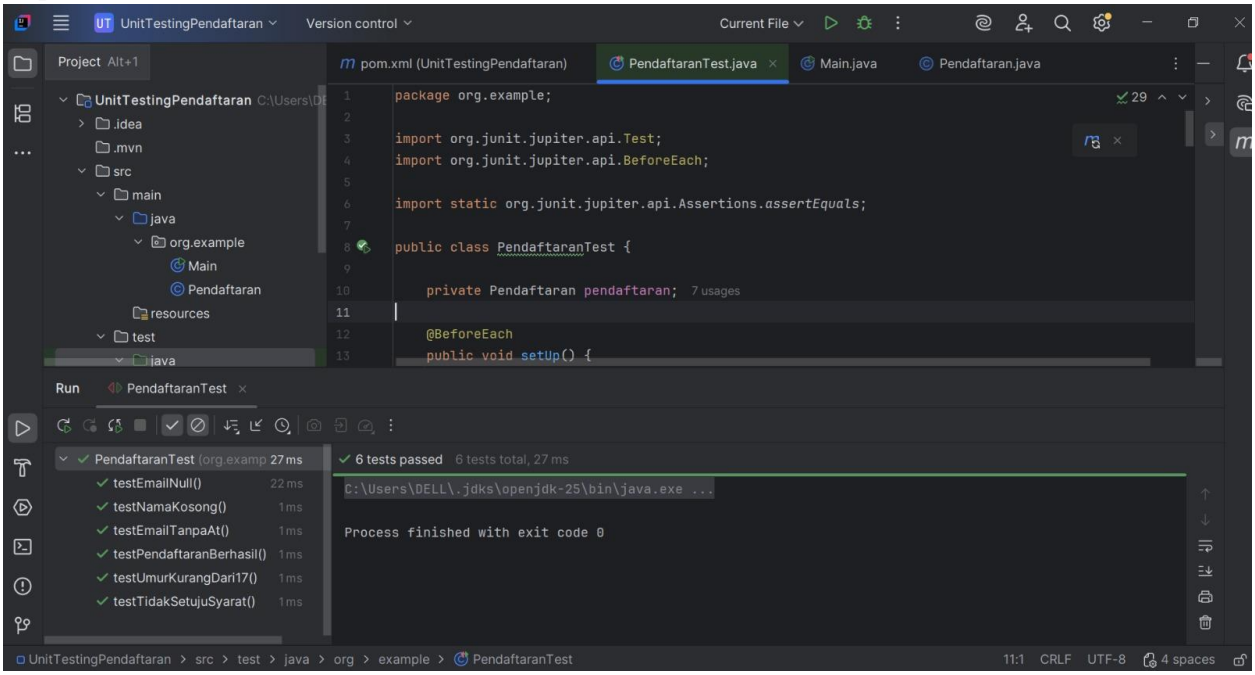
Bagian B : Unit Testing Framework

Soal 3

1. Tuliskan 5 test case berdasarkan independent path yang telah Anda buat di Minggu 3!

Path	Input	Expected output	Kondisi
Path 1	nama="", umur=21, email="cindyyulianti27@gmail.com", setujuSyarat=true	"Nama harus diisi"	nama kosong/null
Path 2	nama="Nazwa", umur=11, email="Nazwasalsabila@gmail.com", setujuSyarat=true	"Umur minimal 17 tahun"	nama valid, umur < 17
Path 3	nama="Cindy", umur=21, email=null, setujuSyarat=true	"Email tidak valid"	nama valid, umur ≥ 17, email null
Path 4	nama="Cindy", umur=21, email="cindyyulianti.gmail.com", setujuSyarat=true	"Email tidak valid"	nama valid, umur ≥ 17, email tidak mengandung @
Path 5	nama="Cindy", umur=21, email="cindyyulianti27@gmail.com", setujuSyarat=false	"Harus menyetujui syarat dan ketentuan"	nama valid, umur ≥ 17, email valid, tidak setuju syarat
Path 6	nama="Cindy", umur=21, email="cindyyulianti27@gmail.com", setujuSyarat=true	"Pendaftaran berhasil"	nama valid, umur ≥ 17, email valid, setuju syarat

2. Implementasikan test case tersebut ke dalam kode unit test JUnit!



```
package org.example;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.BeforeEach;
import static org.junit.jupiter.api.Assertions.assertEquals;

public class PendaftaranTest {
    private Pendaftaran pendaftaran;
    @BeforeEach
    public void setUp() {
        pendaftaran = new Pendaftaran();
    }
    // Path 1: Nama kosong / null
    @Test
    public void testNamaKosong() {
        assertEquals(
                "Nama harus diisi",
                pendaftaran.validasiPendaftaran(
                        "", 21, "cindyyulianti27@gmail.com", true
                )
        );
    }
    // Path 2: Nama valid, umur < 17
    @Test
    public void testUmurKurangDari17() {
        assertEquals(
```

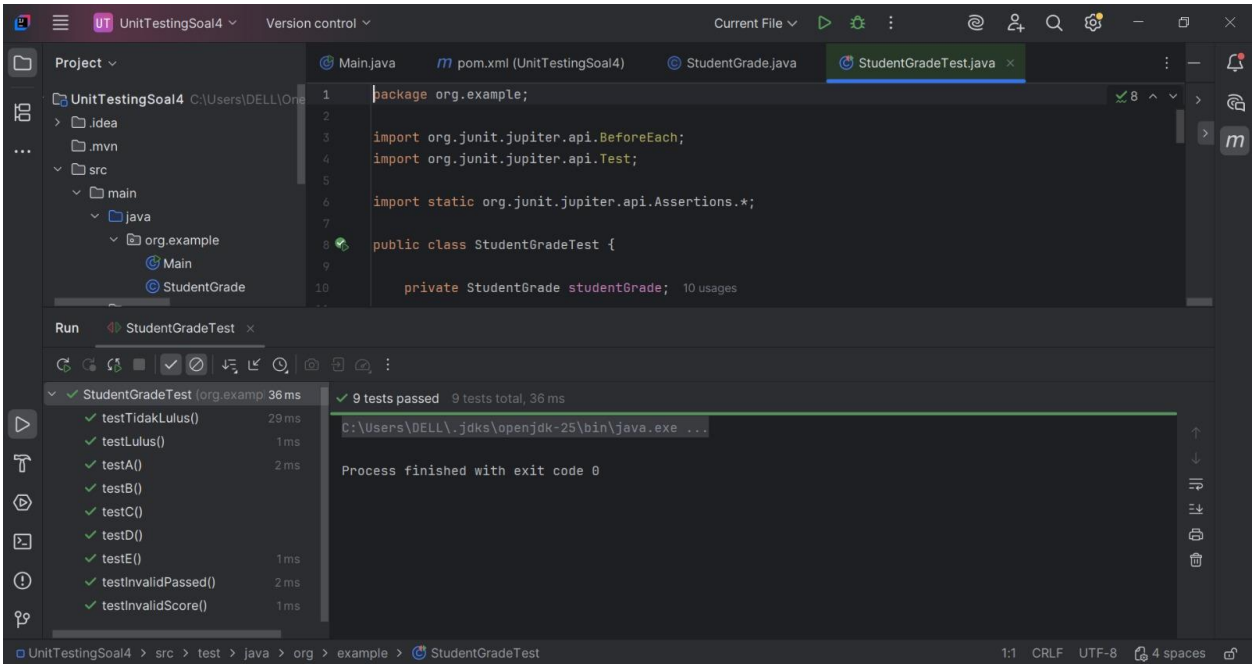
```
                "Umur minimal 17 tahun",
                pendaftaran.validasiPendaftaran(
                        "Nazwa",  11,  "Nazwasalsabila@gmail.com",
true
                )
        );
    }
    // Path 3: nama valid, umur ≥ 17, email null
    @Test
    public void testEmailNull() {
        assertEquals(
                "Email tidak valid",
                pendaftaran.validasiPendaftaran(
                        "Cindy", 21, null, true
                )
        );
    }
    // Path 4: nama valid, umur ≥ 17, email tidak mengandung @
    @Test
    public void testEmailTanpaAt() {
        assertEquals(
                "Email tidak valid",
                pendaftaran.validasiPendaftaran(
                        "Cindy",  21,  "cindyyulianti.gmail.com",
true
                )
```

```
        );
    }
    // Path 5: nama valid, umur ≥ 17, email valid, tidak setuju
syarat
    @Test
    public void testTidakSetujuSyarat() {
        assertEquals(
                "Harus menyetujui syarat dan ketentuan",
                pendaftaran.validasiPendaftaran(
                        "Cindy", 21, "cindyyulianti27@gmail.com",
false
                )
        );
    }
    // Path 6: Pendaftaran berhasil
    @Test
    public void testPendaftaranBerhasil() {
        assertEquals(
                "Pendaftaran berhasil",
                pendaftaran.validasiPendaftaran(
                        "Cindy", 21, "cindyyulianti27@gmail.com",
true
                )
        );
    }
}
```

Soal 4

1. Hitung Cyclomatic Complexity dari method calculateGrade()!
   - CC = E – N + 2P
     CC = 17 – 13 + 2P = 6
   - CC = Predicate +1
     CC = 5 + 1 = 6

2. Tentukan Independent Path dari method calculateGrade()!
   1) P1: 1-2-3-13
      (skor kurang dari 0 atau lebih dari 100)
   2) P2: 1-2-4-5-13
      (skor valid dan skor ≥ 80) → return "A"
   3) P3: 1-2-4-6-7-13
      (skor valid dan skor < 80 tetapi ≥ 70) → return "B"
   4) P4: 1-2-4-6-8-9-13
      (skor valid dan skor < 70 tetapi ≥ 60) → return "C"
   5) P5: 1-2-4-6-8-10-11-13
      (skor valid dan skor < 60 tetapi ≥ 50) → return "D"
   6) P6: 1-2-4-6-8-10-12-13
      (skor valid dan skor < 50) → return "E"
3. Implementasikan unit test JUnit untuk kedua method (calculateGrade dan isPassed)!



```
package org.example;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

public class StudentGradeTest {

    private StudentGrade studentGrade;

    @BeforeEach
    public void setUp() {
        studentGrade = new StudentGrade();
    }

    // P1: Skor tidak valid
    @Test
    public void testInvalidScore() {
        assertThrows(IllegalArgumentException.class,
                () -> studentGrade.calculateGrade(-1));
    }

    // P2: Grade A
    @Test
    public void testA() {
        assertEquals("A", studentGrade.calculateGrade(80));
    }

    // P3: Grade B
    @Test
    public void testB() {
        assertEquals("B", studentGrade.calculateGrade(70));
    }

    // P4: Grade C
    @Test
    public void testC() {
        assertEquals("C", studentGrade.calculateGrade(60));
    }
```

```
    // P5: Grade D
    @Test
    public void testD() {
        assertEquals("D", studentGrade.calculateGrade(50));
    }

    // P6: Grade E
    @Test
    public void testE() {
        assertEquals("E", studentGrade.calculateGrade(49));
    }

    // Q1: Skor tidak valid pada isPassed()
    @Test
    public void testInvalidPassed() {
        assertThrows(IllegalArgumentException.class,
                () -> studentGrade.isPassed(-1));
    }

    // Q2: Mahasiswa lulus
    @Test
    public void testLulus() {
        assertTrue(studentGrade.isPassed(60));
    }

    // Q3: Mahasiswa tidak lulus
    @Test
    public void testTidakLulus() {
        assertFalse(studentGrade.isPassed(59));
    }
}
```

Bagian C : Github Actions

Soal 5

1. Apa nama workflow di atas?
   name: Python CI.
2. Trigger/event apa saja yang memicu workflow?
   - Push: Ketika terdapat perubahan kode yang di-push ke branch main atau develop.
   - Pull Request: Ketika terdapat pull request yang ditujukan ke branch main.
3. Apa fungsi dari strategy matrix pada workflow di atas?
   Strategy matrix berfungsi untuk menjalankan pengujian secara otomatis menggunakan beberapa versi Python, yaitu Python 3.10, 3.11, dan 3.12. Dengan demikian, pengujian dilakukan pada ketiga versi tersebut sehingga dapat diketahui apakah kode program berjalan dengan baik dan kompatibel pada setiap versi Python.
4. Sebutkan 5 steps yang ada di dalam job test!
   Berdasarkan kode tersebut hanya terdapat 4 steps dalam job test :
   1) Checkout code
   2) Setup Python
   3) Install dependencies
   4) Run tests

Soal 6

Membuat Workflow YAML:

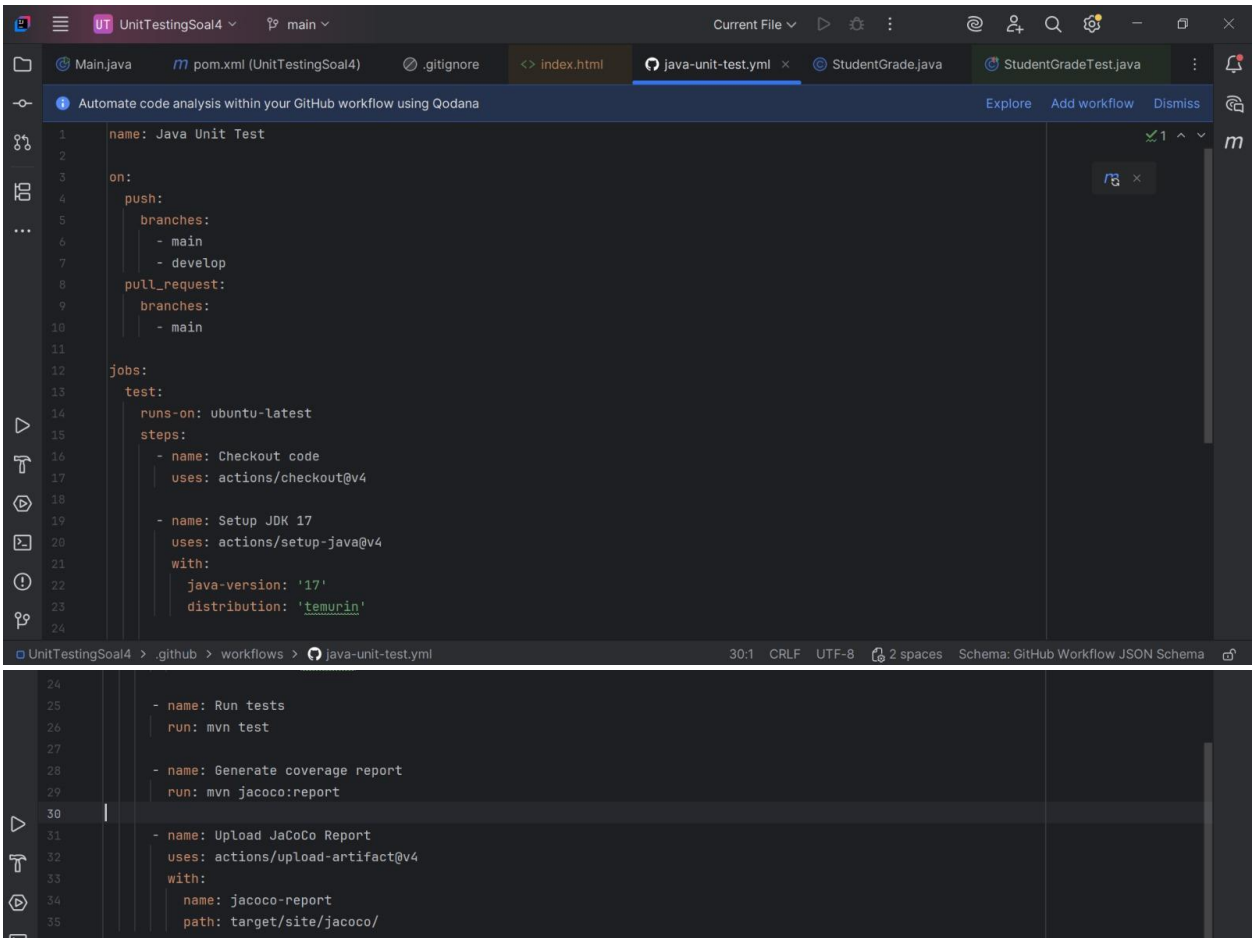
```
name: Java Unit Test

on:
  push:
    branches:
      - main
      - develop
  pull_request:
    branches:
      - main

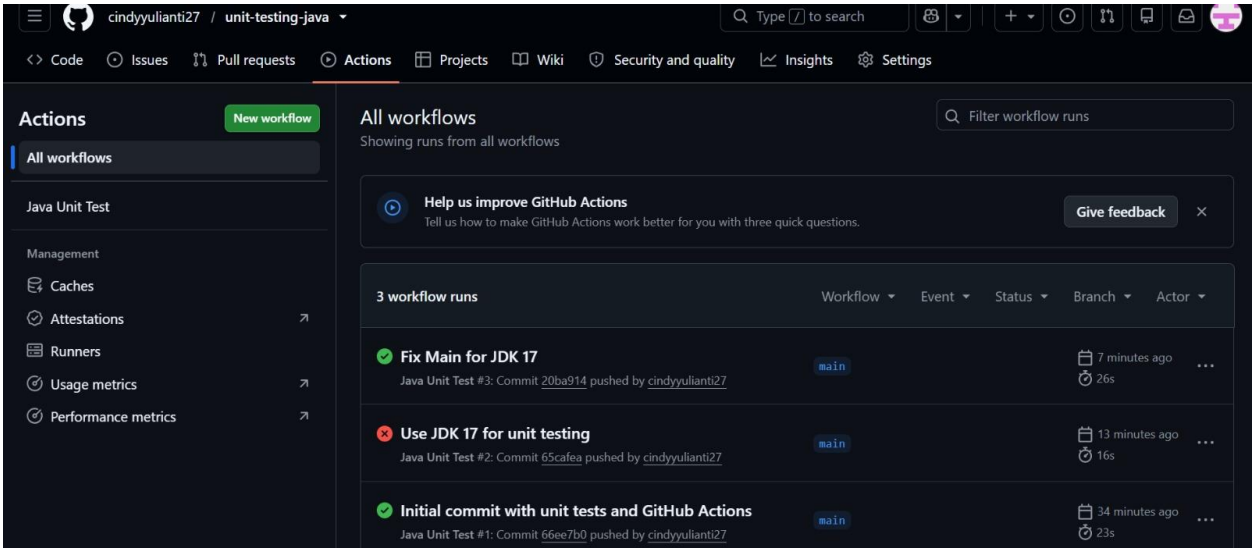
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Setup JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
      - name: Run tests
        run: mvn test
```

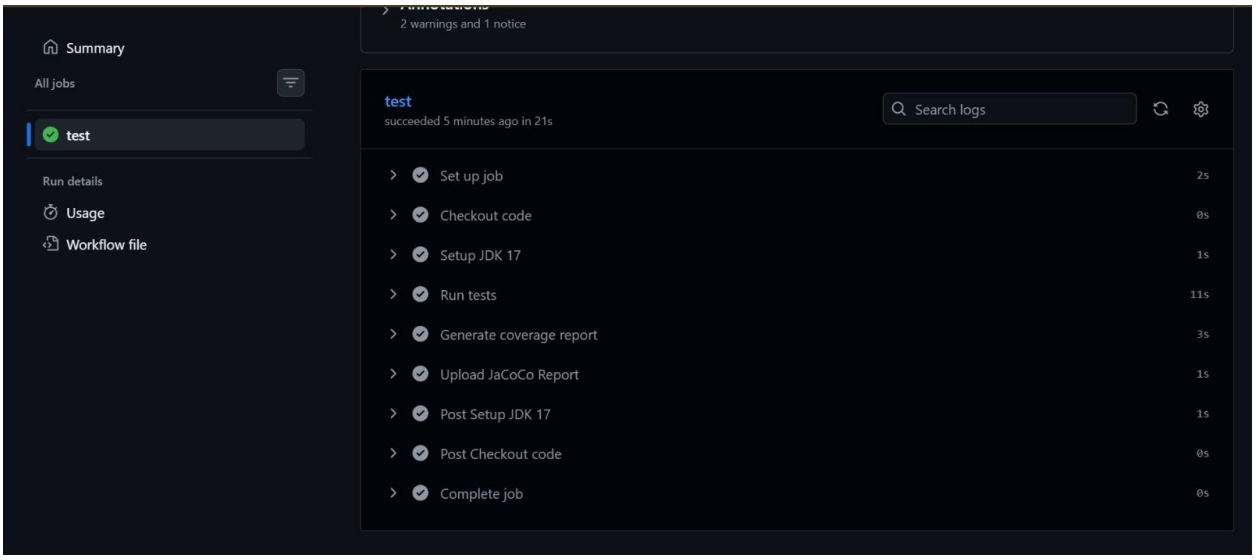
```
    - name: Generate coverage report
      run: mvn jacoco:report
    - name: Upload JaCoCo Report
      uses: actions/upload-artifact@v4
      with:
        name: jacoco-report
        path: target/site/jacoco/
```

a. Kode Workflow GitHub Actions

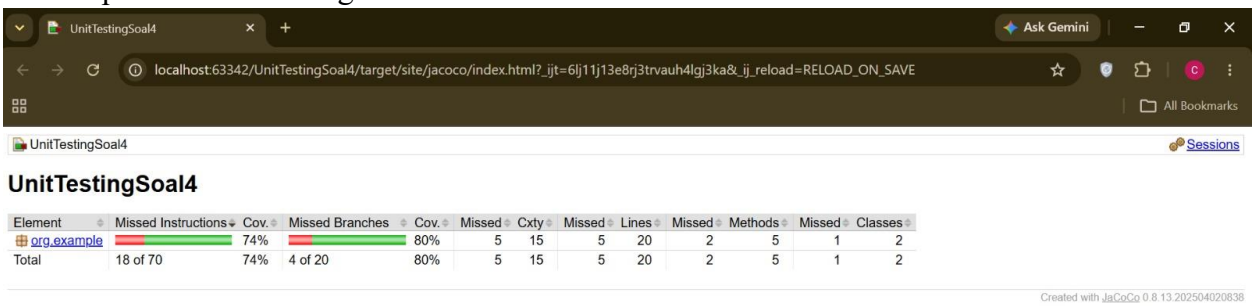


b. Hasil Eksekusi workflow





c. Hasil laporan code coverage JaCoCo



Bagian D : Code Coverage & Integrasi

Soal 7

1. Hitung total statement yang ada di project!
   Total statement yang ada di project adalah 50 statement

2. Hitung total statement yang tidak tereksekusi (Miss)!
   Total statement yang tidak ter-cover adalah 11 statement

3. File mana yang memiliki coverage paling rendah? Berapa persen?
   File yang memiliki coverage paling rendah adalah src/payment.py, dengan tingkat coverage sebesar 60%

4. Apa yang harus dilakukan untuk meningkatkan coverage pada file payment.py?
   Untuk meningkatkan coverage pada file payment.py, perlu ditambahkan test case menggunakan pytest untuk menguji bagian kode yang belum ter-cover. Pengujian dapat mencakup kondisi pembayaran berhasil, pembayaran gagal, saldo tidak mencukupi, nominal pembayaran tidak valid, dan kondisi error lainnya. Setelah itu, jalankan kembali pengujian dan periksa laporan coverage untuk memastikan lebih banyak statement telah diuji.

5. Menurut Anda, apakah coverage 78% sudah cukup baik? Jelaskan alasannya!
   Menurut saya, statement coverage sebesar 78% cukup baik, tetapi masih perlu ditingkatkan karena belum mencapai target 80–90%. Dari total 50 statement, masih terdapat 11 statement yang belum teruji. Hal ini meningkatkan risiko adanya kesalahan yang tidak terdeteksi, terutama pada proses pembayaran yang harus menangani berbagai kondisi, seperti pembayaran berhasil, gagal, nominal tidak valid, atau kesalahan input. Oleh karena itu, pengujian perlu diperluas, khususnya pada bagian pembayaran yang memiliki coverage rendah. Pengujian juga tidak sebaiknya hanya berfokus pada pencapaian persentase coverage, tetapi harus memastikan bahwa setiap fungsi penting berjalan sesuai harapan dan mampu menangani kondisi normal maupun kesalahan. Dengan demikian, keandalan aplikasi dapat meningkat dan risiko kesalahan saat digunakan dapat dikurangi.

Link GitHub :

https://github.com/cindyyulianti27/latihan-git-1224606

https://github.com/cindyyulianti27/unit-testing-java